Practical 5

Aim : Study and understand the basics of Solidity language.

Introduction :

Solidity is a high-level, statically typed programming language used mainly for writing smart contracts on blockchain platforms such as Ethereum. Smart contracts are programs stored on a blockchain that automatically execute predefined instructions when their conditions are met.

Solidity has syntax similar to languages such as C++, Java, and JavaScript. It supports programming concepts such as variables, functions, arrays, mappings, structures, events, and constructors.

Basic Concepts of Solidity :

1. Contract

A contract is the main structure of a Solidity program. It contains variables and functions.

```
contract Student {
   // Variables and functions
}
```

2. Data Types

Solidity provides different data types for storing information.

- uint – Stores unsigned integers.
- int – Stores positive and negative integers.
- string – Stores text.
- bool – Stores true or false.
- address – Stores a blockchain address.
- bytes – Stores binary data.

Example:

```
string public name;
uint public age;
bool public passed;
```

3. State Variables

State variables are declared inside a contract and their values are stored on the blockchain.

```
uint public age = 20;
```

4. Functions

Functions are used to perform operations or access contract data.

```
function setAge(uint _age) public {
   age = _age;
}
```

The public keyword allows the function to be called from outside the contract.

5. Function Visibility

Solidity provides different visibility levels:

- public – Can be accessed from anywhere.
- private – Can be accessed only inside the same contract.
- internal – Can be accessed inside the contract and derived contracts.
- external – Mainly called from outside the contract.

6. View and Pure Functions

A view function can read blockchain data but cannot modify it.

```
function getAge() public view returns (uint) {
   return age;
}
```

A pure function does not read or modify blockchain state.

```
function add(uint a, uint b) public pure returns (uint) {
   return a + b;
}
```

7. Mapping

A mapping stores data in the form of key-value pairs.

```
mapping(address => uint) public balances;
```

Here, a blockchain address can be associated with a particular balance.

8. Array

An array is used to store multiple values of the same type.

```
uint[] public numbers;
```

Values can be added using:

```
numbers.push(10);
```

9. Struct

A structure allows related information to be grouped together.

```
struct Student {
    string name;
    uint age;
}
```

10. Constructor

A constructor is executed only once, when the smart contract is deployed.

```
constructor(string memory _name) {
    name = _name;
}
```

11. msg.sender

msg.sender represents the blockchain address that called the current function.

```
function getAddress() public view returns (address) {
    return msg.sender;
}
```

12. Events

Events are used to record information in blockchain transaction logs.

```
event StudentAdded(string name);
```

An event can be triggered using:

```
emit StudentAdded(name);
```

Program :

The following Solidity program creates a simple student smart contract that stores and displays student information.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
contract Student {
   string public name;
   uint public age;
   function setStudent(string memory _name, uint _age) public {
      name = _name;
      age = _age;
   }
   function getStudent() public view returns (string memory, uint) {
      return (name, age);
   }
}
```

Explanation of the Program

- SPDX-License-Identifier specifies the license of the source code.
- pragma solidity ^0.8.0 specifies the Solidity compiler version.
- contract Student creates a smart contract named Student.
- name stores the student's name.
- age stores the student's age.
- setStudent() is used to store the student's details.
- getStudent() is used to retrieve the stored details.
- memory indicates that the string is temporarily stored during function execution.
- view indicates that getStudent() only reads the stored data.

Expected Output

After deploying the contract and calling setStudent() with:

```
Name: Divy
Age: 20
Calling getStudent() returns:
Divy
20
```

Result

Thus, the basic concepts of Solidity programming language were studied and understood. A simple Student Smart Contract was successfully created to store and retrieve student information on a blockchain.